# Retrieval-Augmented Few-Shot Prompting vs. LoRA Fine-Tuning for Issue Report Classification: An Empirical Study

**Anonymous author**
Anonymous affiliation

## Abstract

**Background.** Issue report classification (IRC) accelerates modern software maintenance. State-of-the-art automated IRC methods fine-tune Large Language Models (LLMs) with Low-Rank Adaptation (LoRA), which is computationally expensive and must be repeated to incorporate newly labeled issue reports or whenever the model is swapped.

**Aims.** We empirically investigate whether retrieval-augmented few-shot prompting offers a practical alternative to LoRA fine-tuning for IRC, and how much of its performance comes from the LLM versus retrieval alone.

**Method.** Given a query issue, we retrieve its top-$k$ nearest neighbors from an index of classified issues, and propose three methods: (i) VOTAG, a similarity-weighted vote on the neighbors' labels, (ii) RAGTAG, which presents the neighbors as classified examples in a few-shot prompt to an LLM, and (iii) BRAGTAG, a RAGTAG variant with a more balanced retrieval scheme. We evaluate all three against a LoRA fine-tuning baseline on an 11-project benchmark, in both *project-specific* (PS) and *project-agnostic* (PA) data scope settings. Experiments use Qwen2.5-Instruct at four sizes.

**Results.** Every RAGTAG configuration outperforms VOTAG's best result. In PS, RAGTAG outperforms fine-tuning by 0.021–0.040 macro $F_1$ across all model sizes. Fine-tuning PA beats RAGTAG PS by only 0.029 aggregate macro $F_1$ despite 11× more labeled data. BRAGTAG PS closes the gap to TOST equivalence at $\delta$=0.02, tightening to statistical equivalence at $\delta$=0.01 with VOTAG fallback for invalid LLM outputs. Averaged across model sizes, RAGTAG and BRAGTAG require 33% less peak GPU memory than fine-tuning.

**Conclusions.** Retrieval-augmented few-shot prompting is a practical alternative to LoRA fine-tuning for IRC, particularly when hardware resources are constrained, labeled data is limited, or the deployment has frequent model or data updates.

**2012 ACM Subject Classification** Software and its engineering → Software notations and tools; Computing methodologies → Natural language processing

**Keywords and phrases** issue classification, retrieval-augmented generation, fine-tuning, large language models, mining software repositories

**Digital Object Identifier** 10.4230/LIPIcs...

## 1 Introduction

Issue Tracking Systems provide a structured way to communicate concerns related to the software project by allowing users to submit issue reports [13]. Issues reports can be classified with labels to help with triage. Effective issue report classification (IRC) results in faster issue resolution [33], and is crucial for many software maintenance tasks, such as issue prioritization [10], issue assignment [48, 28], and developer onboarding [39, 41, 5]. However, studies show that labels are rarely applied in issue creation [3, 23, 9, 29]. As a result, project maintainers must manually classify the issues for triage, which is laborious and time-consuming [18].

To address this problem, numerous automated methods have been proposed around supervised learning over three established issue report categories: *bug*, *feature*, and *question* [3, 29, 26, 5, 4]. Early work relied on classic machine learning (ML) techniques [3, 29, 18, 23],


Leibniz International Proceedings in Informatics
LIPICS Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

while more recent studies fine-tune transformer models on labeled issue reports [26, 27, 42, 7, 11, 45, 21]. These approaches require a substantial amount of training data and often struggle with minority classes [26, 27, 11, 22, 43]. The current state-of-the-art methods instruction-tune Large Language Models (LLMs) using Low-Rank Adaptation (LoRA [24]) [22, 5, 4]. However, fine-tuning LLMs is computationally expensive and must be repeated whenever the model is swapped or to incorporate new labeled issues that accumulate in the project [18, 29].

Prior work shows that placing a few demonstration examples in the prompt can boost an LLM's performance on software-related tasks [6, 32] through in-context learning [8]. Since few-shot prompting enhances performance without model training or weight updates, it has the potential to serve as a cost-efficient alternative to fine-tuning LLMs for IRC. Few-shot performance, however, depends on the choice of in-context examples: retrieval-based example selection outperforms random selection [34], and the most similar neighbors of a query tend to share the same label, making them informative demonstrations for classification [14]. The combination of these insights leads to retrieval-augmented few-shot prompting, yet a systematic evaluation of this approach for IRC remains largely underexplored in the literature.

To bridge this gap, we propose **R**etrieval-**A**ugmented **G**enerative **TAG**ging (RAGTAG). Given a query issue, RAGTAG retrieves its top-$k$ most textually similar issues reports from an index of embedded historical issue reports and presents them as classified examples in the prompt. $k$=0 results in a zero-shot LLM baseline, similar to prior work [12]. Our evaluation (Section 5.2) reveals that the *question* class has the weakest RAGTAG performance, and that question issues are frequently misclassified as bugs. We hypothesize that bug-labeled neighbors retrieved for true-question queries reinforce this misclassification. To tackle this problem, we propose **B**alanced **R**etrieval-**A**ugmented **G**enerative **TAG**ging (BRAGTAG), a RAGTAG variant that drops the bug-labeled neighbors whenever their count is within a small margin $m$ of the question-labeled count ($N_{\text{bug}} - N_{\text{question}} \leq m$).

Following the standard $k$-nearest-neighbor classification framework [14], with neighbors weighted by their cosine similarity to the query [17], we propose **Vo**ting-based **TAG**ging (VOTAG). Similar to RAGTAG and BRAGTAG, for a given query issue, VOTAG first retrieves its top-$k$ neighbors, and predicts the query label by a similarity-weighted vote on their labels. VOTAG serves four purposes in this study: First, it establishes a retrieval-only baseline that LLM-based methods must clear to justify their cost. Second, it acts as an ablation that isolates the contribution of the LLM from that of retrieval in RAGTAG and BRAGTAG. Third, its performance as a function of $k$ informs the $k$-grid we evaluate for RAGTAG and BRAGTAG. Finally, it can be used as a fallback mechanism in case of invalid LLM outputs across all methods due to its low cost.

We evaluate our proposed methods against the established LoRA [24] fine-tuning baseline [22, 5, 4] across the Qwen2.5-Instruct model family at 3B, 7B, 14B, and 32B parameters [49] to analyze model scale effect without architecture variance. We use Heo et al.'s [22] 11-project dataset for our experiments and adopt its two evaluation settings: a *project-specific* (PS) configuration where retrieval/training is over each project's individual data split in isolation, and a *project-agnostic* (PA) configuration where retrieval/training is over data from the 11 projects pooled together.

Results show that retrieved few-shot examples consistently improve IRC performance. Even a single retrieved example outperforms the zero-shot baseline across all model sizes (Section 5.2). LLM reasoning also consistently improves over pure retrieval. Every RAGTAG configuration outperforms VOTAG's best result. Notably, however, VOTAG comes within 0.018 macro $F_1$ of Qwen-3B's zero-shot (Section 5.1).

Additionally, retrieval-augmented few-shot prompting shows competitive performance to fine-tuning. In the PS setting, RAGTAG outperforms fine-tune by 0.021–0.040 macro $F_1$ on all model sizes (Section 5.4). Fine-tune only catches up when given 11× more labeled data via cross-project pooling (PA), and even then RAGTAG remains competitive: aggregated across all model sizes, fine-tune PA leads RAGTAG PS by 0.029 macro $F_1$, and the two methods are statistically tied at Qwen-3B and Qwen-32B. BRAGTAG closes this gap by reducing the question→bug misclassification rate without sacrificing performance on other labels (Section 5.3). The aggregate ($BRAGTAG-$ fine-tune) difference is $-0.010$ (bootstrap 95% CI $[-0.018, -0.003]$), passing TOST equivalence at $\delta$=0.02. Applying a VOTAG fallback to invalid LLM outputs across all methods tightens this to statistical equivalence, passing TOST at $\delta$=0.01 (bootstrap 95% CI $[-0.007, +0.008]$) with no additional cost.

Retrieval-augmented few-shot offers deployment advantages as well. Averaged across the four model sizes, RAGTAG and BRAGTAG require 33% less peak GPU memory than fine-tuning (Section 5.4). Additionally, incorporating newly labeled issues and swapping the underlying model mid-production do not require retraining cycles.

Overall, this study demonstrates retrieval-augmented few-shot's practicality in IRC, particularly when hardware resources are constrained, labeled data is limited, or the deployment requires frequent model or data updates.

The remainder of this paper is organized as follows: Section 2 reviews prior work. Section 3 explains our proposed methods and the fine-tuning baseline. Section 4 details the experimental setup. Section 5 explains our evaluations and reports the results. Section 6 discusses failure modes, practical implications, and future work. Section 7 addresses the potential threats to validity. Finally, Section 8 concludes the paper.

## 2 Related Work

Early IRC studies employed data mining and classic ML classifiers [3, 29, 18]. For example, Antoniol et al. [3] leveraged a naive Bayes classifier to distinguish bugs from other issue reports. However, these methods yielded limited performance due to their shallow semantic understanding of issue report text [22].

To overcome this limitation, more recent work has adopted BERT-like transformer models for their stronger contextual understanding [11, 7, 42, 26, 27]. These studies typically fine-tune the models on historical issues in a supervised setting. For instance, Izadi et al. [26] fine-tuned a RoBERTa [36] model to classify issues into *Bugs*, *Features*, or *Questions*. Similarly, Trautsch et al. [42] trained a seBERT [44] model to classify issues into the same three categories. Although transformer-based approaches achieve strong results, they rely on large amounts of training data, which hinders their generalization to projects with few labeled issues [22, 26]. They also struggle to accurately classify minority classes that have fewer training examples [27, 11].

These challenges have motivated the exploration of LLMs, which exhibit strong zero-shot text classification capabilities thanks to extensive pretraining on massive corpora [22, 5, 4, 8, 47, 13]. State-of-the-art LLM-based methods achieve their best performance by fine-tuning the models with Low-Rank Adaptation (LoRA [24]) on labeled issue datasets [22, 5, 4]. However, fine-tuning remains computationally expensive and must be repeated whenever the underlying model is swapped or as newly labeled issues accumulate in the project [18, 29].

In this study, we conduct an empirical evaluation of retrieval-augmented few-shot prompting to assess whether it is a more cost-efficient and practical alternative to fine-tuning LLMs for IRC.

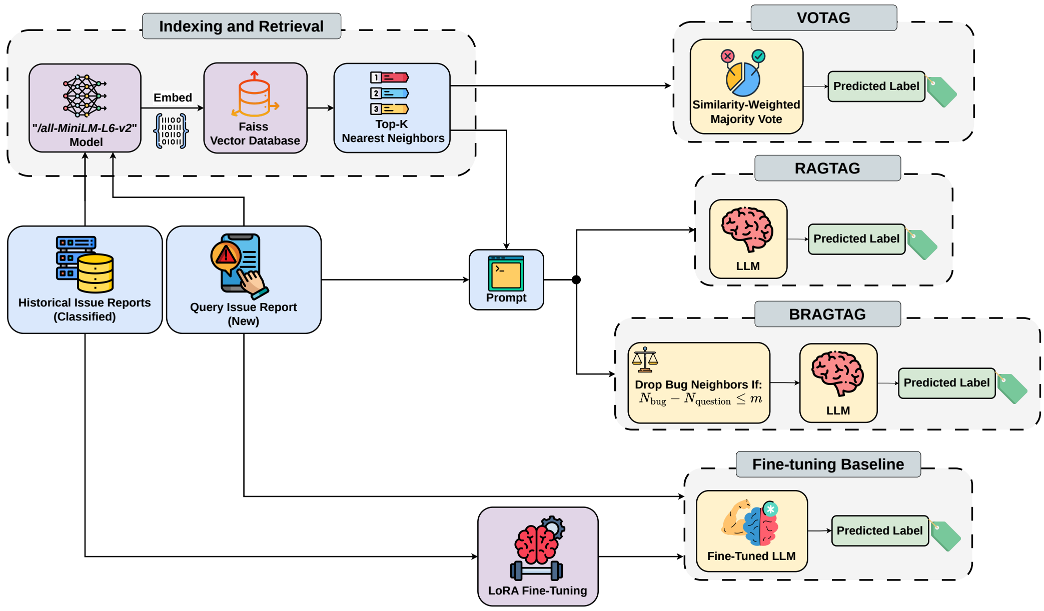


**Figure 1** Overview of the four IRC approaches evaluated in this paper. All approaches share the same embedding and retrieval pipeline (Section 3.2); they differ in how the retrieved neighbors and the query issue are converted into a label.

# 3 Approach

Figure 1 provides an overview of the four approaches evaluated in this paper: RAGTAG, BRAGTAG, VOTAG, and the fine-tuning baseline.

## 3.1 Problem Formulation

Given a query issue report, we classify it into only one of the three labels: *bug*, *feature*, or *question*. Only the issue's title and description text are used for classification with no other signals from the issue tracker. The defined label space is adopted directly from prior IRC studies [4, 5, 26, 29, 3].

## 3.2 Indexing and Retrieval

To prepare the issues for retrieval based on textual similarity, we first embed their textual content (i.e., *title* + *description*). We do not embed the issue labels. Instead, we retain them as metadata associated with each embedded issue. To generate the embeddings, we use the `all-MiniLM-L6-v2` model from the Sentence-Transformers library [38]. We selected this model due to its lightweight architecture, fast performance, and effectiveness on issue report text [25]. The embeddings are then indexed using the Faiss vector database library [16], which provides efficient high-dimensional similarity retrieval. At the retrieval stage, the query issue is first embedded with the same model. Next, Faiss calculates the cosine similarity of indexed issue reports to the query, and returns the top-K nearest neighbors, ordered by descending similarity.

## 3.3 RAGTAG

RAGTAG is our proposed retrieval-augmented few-shot prompting approach for IRC. RAGTAG uses the retrieval pipeline described in Section 3.2 to retrieve the top-K nearest neighbors to the query issue, and present them as classified examples to the LLM in a few-shot prompt. This enables a more accurate classification, based on in-context learning [8]. The choice to present the top-K nearest neighbors as examples is based on two insights from prior research: First, retrieval-based example selection outperforms random selection [34]. Second, the most similar neighbors tend to share the same labels [14]; therefore, they are informative for the classification.

We create the few-shot prompt in three parts. First, the system message frames the classification task and the label space. Second, the top-K nearest neighbors are listed in retrieval order (descending similarity), each formatted as *title + description + label*. Finally, the query issue is appended to the end of the prompt, ensuring it receives high attention from the LLM [35]. $K = 0$ produces a zero-shot prompting baseline, similar to prior work [12].

We evaluate RAGTAG at top-K values in $\{0, 1, 3, 6, 9, 12, 15\}$. Details for the K value grid selection are provided in Sections 5.1 and 5.2. The maximum sequence length is set to 8,192 tokens.[1] Roughly 30% of the context quota is reserved for the query issue and 70% for the neighbors. If the neighbors exceed their overall quota, they are truncated proportionally based on their length (longer neighbors get more truncation). The query issue gets truncated if it exceeds its budget as well. Truncation happens on the right side of the description text, preserving all titles and labels (if any).

The LLM is instructed to generate the predicted label wrapped in XML tags (e.g. `<label>bug</label>`). The labels of the presented examples also follow the same format. This technique enables easy detection of the predicted label from the LLM's generation, and reduces variance in LLM outputs caused by prompt format sensitivity [40]. We use a regular expression to extract the predicted label. Unparseable outputs are marked as *invalid*.

## 3.4 Balanced RAGTAG (BRAGTAG)

BRAGTAG is a RAGTAG variant designed to reduce the frequent question→bug misclassifications, without sacrificing performance on the other two labels (Section 5.2). We hypothesize that bug-labeled neighbors retrieved for true-question queries reinforce the question→bug misclassification. Therefore, before prompt construction, BRAGTAG inspects the top-K neighbors and drops the bug-labeled ones when their count is within a small margin $m$ of the question-labeled count ($N_{\text{bug}} - N_{\text{question}} \leq m$). The full misclassification analysis and the empirical justification for BRAGTAG and margin selection are presented in Sections 5.2 and 5.3.

## 3.5 VOTAG

VOTAG is our proposed non-parametric retrieval IRC method. VOTAG uses the retrieval pipeline described in Section 3.2 to retrieve the top-K nearest neighbors of the query issue report, then predicts the final label with a similarity-weighted vote on their labels. This approach follows the standard K-nearest-neighbor classification setup [14], with neighbors

---

[1] Maximum sequence length is selected empirically. 8,192 tokens offered the best trade-off between classification accuracy and inference speed across our preliminary experiments.

weighted by their similarity to the query [17].[2] Formally, for a query issue $q$ with retrieved neighbors $\{(n_i, l_i, s_i)\}_{i=1}^{K}$, where $n_i$ is the $i$-th nearest neighbor with label $l_i$ and cosine similarity $s_i$ to $q$, the label is predicted by:

$$\hat{y} = \arg\max_{c \in \mathcal{L}} \sum_{i=1}^{K} s_i \cdot \mathbf{1}[l_i = c], \tag{1}$$

where $\mathcal{L}$ is the label set and $\mathbf{1}[\cdot]$ is the indicator function. VOTAG breaks ties by returning the label of the most similar neighbor in the tied classes.

## 3.6 Fine-Tuning

SOTA IRC approaches have achieved strong performance by supervised fine-tuning of LLMs [22, 5, 4], using Low-Rank Adaptation (LoRA) [24]. In these studies, the model is trained on labeled issue reports and predicts the labels for unseen test issues. Each training example is formatted as an instruction prompt which includes the issue's title and description as input, and its label as output. At inference, the trained model receives the test issue in the same format, and predicts the empty label field. Regular expressions are used to extract the prediction from the output, and unparseable outputs are marked as *invalid*. We use this established methodology as our fine-tuning baseline.

The LoRA rank and alpha are set to 16, with a learning rate of $2 \times 10^{-4}$ and the paged AdamW 8-bit optimizer. Training runs for 3 epochs with a per-device batch size of 1 and gradient accumulation steps of 16. The max sequence length is set to 2,048 tokens, which covers 94.9% of the issues in our dataset. Issues that exceed this budget get truncated from the right side of their description text to fit.

# 4 Experimental Setup

## 4.1 Dataset and Settings

We use the dataset introduced in Heo et al. [22] for our experiments. The dataset has a total of 6,600 issue reports gathered from eleven active open-source projects: `ansible/ansible`, `bitcoin/bitcoin`, `dart-lang/sdk`, `dotnet/roslyn`, `facebook/react`, `flutter/flutter`, `kubernetes/kubernetes`, `microsoft/TypeScript`, `microsoft/vscode`, `opencv/opencv`, and `tensorflow/tensorflow`. Each project has 600 issue reports with an equal distribution across the labels (*bug, feature, question*). To support their LLM fine-tuning method, Heo et al. divided the dataset into 50/50 train and test splits, stratified by project and label. This results in 300 train and 300 test issue reports per project.

We also adopt Heo et al.'s project-specific (PS) and project-agnostic (PA) configurations to examine how data scope affects IRC performance. In the PS setting, each project's train and test data are used in isolation, resulting in eleven different train-test pairs. In the PA setting, the train splits of all projects are merged into a single training set of 3,300 issue reports. In both configurations, RAGTAG and VOTAG only use the training splits as their retrieval index to ensure fair comparison with the fine-tuning baseline. Figure 2 illustrates the difference between the two settings.

[2] Squared-similarity weighting and uniform majority were also evaluated, but were outperformed by similarity-weighted voting.

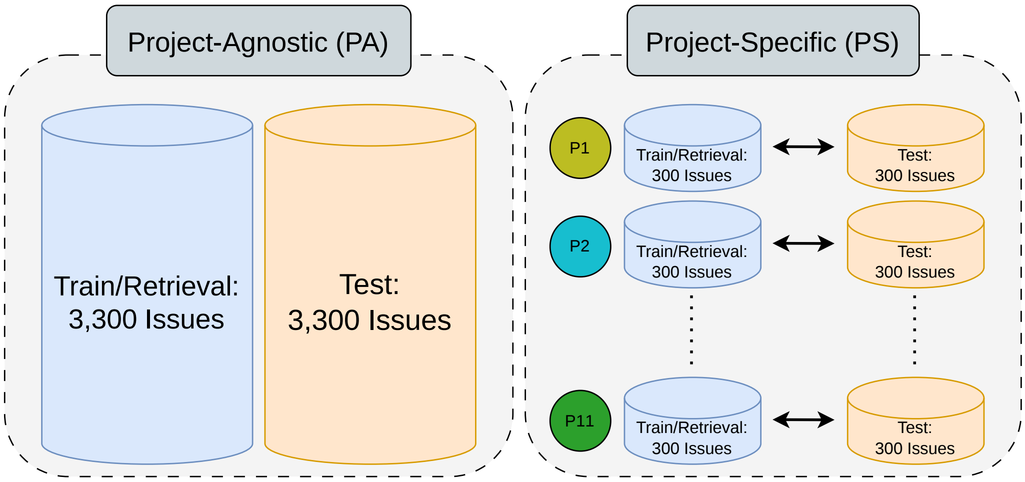


**Figure 2** Project-Specific (PS) vs. Project-Agnostic (PA) settings. PS uses one separate retrieval index per project (300 train issues each); PA merges all 11 projects' train splits into a single 3,300-issue index.

## 4.2 Model Zoo

We use the Qwen2.5-Instruct model family [49] in our evaluations with four different parameter sizes: 3B, 7B, 14B, and 32B. Qwen2.5 is an open-source LLM with strong performance on general-purpose benchmarks. Using the same model at different parameter sizes isolates the effect of model scale from model architecture across LLM-based IRC approaches. For memory efficiency the models are loaded in 4-bit quantization with Unsloth[3]. Temperature is set at 0.1 for deterministic classifications.

## 4.3 Evaluation Metrics

We report the per-class $F_1$ and the macro-averaged $F_1$ over the three classes (`bug`, `feature`, `question`). The macro $F_1$ is calculated over the total 3,300-issue test set in all settings. We also report the rate of outputs that fail to parse to a valid label (*invalid rate*). For statistical significance checks, we report paired bootstrap 95% confidence intervals on the macro $F_1$ differences between methods.

## 4.4 Hardware Setup

The experiments were conducted on a machine with one NVIDIA L40 GPU (48 GB VRAM), 8 allocated CPU cores from an Intel Xeon Scalable processor, and 64 GiB of system RAM.

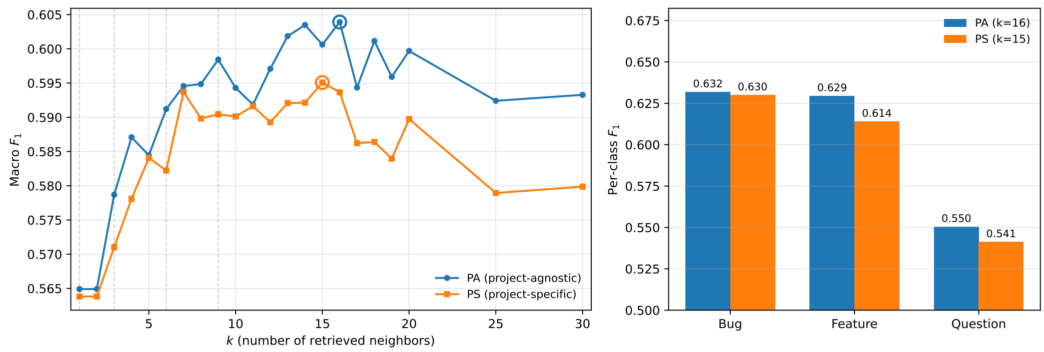


**Figure 3 (left)** VOTAG macro $F_1$ as a function of $k$ in both PS and PA settings. Circles mark peak performance. **(right)** Per-class $F_1$ at each setting's best $k$ ($k$=15 for PS, $k$=16 for PA).

# 5 Evaluations

## 5.1 RQ1. To what extent can similarity-weighted voting on the nearest neighbors classify issue reports, and at what point does adding neighbors stop helping?

We evaluate VOTAG's performance on both PA and PS settings at $k$ values 1 to 30. The resulting macro F1 scores are shown in Figure 3. VOTAG's best macro F1 is 0.595 ($k$=15) in PS and 0.604 ($k$=16) in PA. Both settings reach 99.8% (PS) and 98.5% (PA) of their respective peak by $k$=7. When adding neighbors beyond the peak, PS loses 0.015 and PA loses 0.011 by $k$=30.

PA consistently outperforms PS at every $k$. However, the performance gap is marginal: averaging 0.007 macro F1, with a maximum of 0.015 at $k$=18. This similar performance is due to the large overlap of the top-$k$ neighbors in both settings. Specifically, PA and PS share 84–90% of the top-$k$ neighbors across $k \in [3, 30]$. Therefore, even when using the full 3,300-issue retrieval index, most of the retrieved neighbors are from the same project. This means issue reports from the same project tend to be closer in the embedding space.

Question consistently has the weakest performance among the three labels at every $k$ in both settings. At best $k$, PS per-class F1 is 0.630 for bug, 0.614 for feature, and 0.541 for question. The corresponding PA values are 0.632, 0.629, and 0.550 (Figure 3). Additionally, questions are more frequently misclassified as bugs: 32% of question issues are predicted as bugs in both settings, while 18% (PS) and 17% (PA) are predicted as features. Since VOTAG predicts the label with the most accumulated similarity weight, this misclassification asymmetry indicates that question issues are more similar to bugs than to features in the embedding space.

## 5.2 RQ2. How well does retrieval-augmented few-shot prompting classify issue reports across model scales, and how does performance vary with the number of in-context examples?

We evaluate RAGTAG at $k$ values $\{0, 1, 3, 6, 9, 12, 15\}$ on both PA and PS settings, where $k$=0 is the zero-shot baseline [12]. $k$ value grid selection is informed by VOTAG's peak

[3] `https://unsloth.ai`

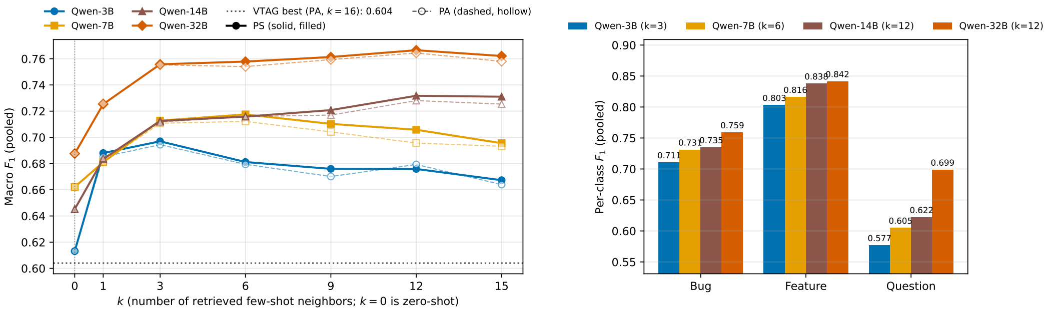


**Figure 4 (left)** RAGTAG macro $F_1$ as a function of $k$ for all four Qwen sizes. $k$=0 is the zero-shot baseline. The dotted horizontal line marks VOTAG's best (0.604, PA $k$=16). **(right)** Per-class $F_1$ for each model size at its best RAGTAG-PS configuration.

performance around $k$=15 in the previous section. Figure 4 (left panel) shows the macro F1 of every model across this grid for both settings.

PS marginally outperforms PA at almost every (model, $k \geq 1$) pair evaluated (zero-shot is identical between settings since no retrieval is performed). The close performance gap is due to the same top-$k$ neighbor overlap observed in the VOTAG analysis. The marginal improvement is significant because PS uses 11$\times$ less retrieval data per project: each PS index covers only its own 300 retrieval issues, while PA uses the full 3,300 across all 11 projects. Given this practical advantage, we conduct the rest of our RAGTAG analysis on the PS setting and report PS numbers throughout this section.

The best $k$ grows with model size: Qwen-3B peaks at $k$=3 (macro F1 = 0.697), Qwen-7B at $k$=6 (0.718), and both Qwen-14B and Qwen-32B at $k$=12 (0.732 and 0.767 respectively). This shows that larger models can use the context from additional examples more effectively. However, adding neighbors past $k$=12 has no benefit and slightly degrades performance: every model loses 0.001–0.015 macro F1 going from $k$=12 to $k$=15.

Every RAGTAG configuration with $k \geq 1$ outperforms the zero-shot baseline. At each model's peak, the gap over zero-shot averages +0.076 macro $F_1$ and ranges from +0.055 (Qwen-7B) to +0.087 (Qwen-14B). This shows that LLMs benefit substantially from the most similar labeled examples for IRC.

Every RAGTAG configuration outperforms VOTAG's best result (0.604 at PA $k$=16), with the smallest margin from Qwen-3B zero-shot (F1 = 0.613, +0.009) and the largest from Qwen-32B PS at $k$=12 (F1 = 0.767, +0.163). These results show that adding the LLM reasoning improves IRC on top of pure retrieval.

Similar to VOTAG, question consistently has the weakest performance among the three labels across all RAGTAG settings. Figure 4 (right panel) shows per-class $F_1$ at each model's best $k$. Even the smallest gaps between question and the other labels, observed at Qwen-32B with $k$=12, are noticeable: question is 0.060 macro $F_1$ below bug and 0.143 below feature. Similar to the VOTAG observations, questions tend to be more frequently misclassified as bugs. At zero-shot, across all four model sizes, 46–59% of true-question issues are predicted as bugs while only 5–16% are predicted as features. RAGTAG narrows this misclassification asymmetry but does not eliminate it: at each model's best $k$, 26–36% of question issues are still predicted as bugs and only 9–15% as features.

These results show that the LLMs are already prone to misclassifying question issues as bugs, even before the top-$k$ examples are used. Therefore, presenting bug-labeled examples for true-question query issues likely helps the misclassification. Based on this intuition, we

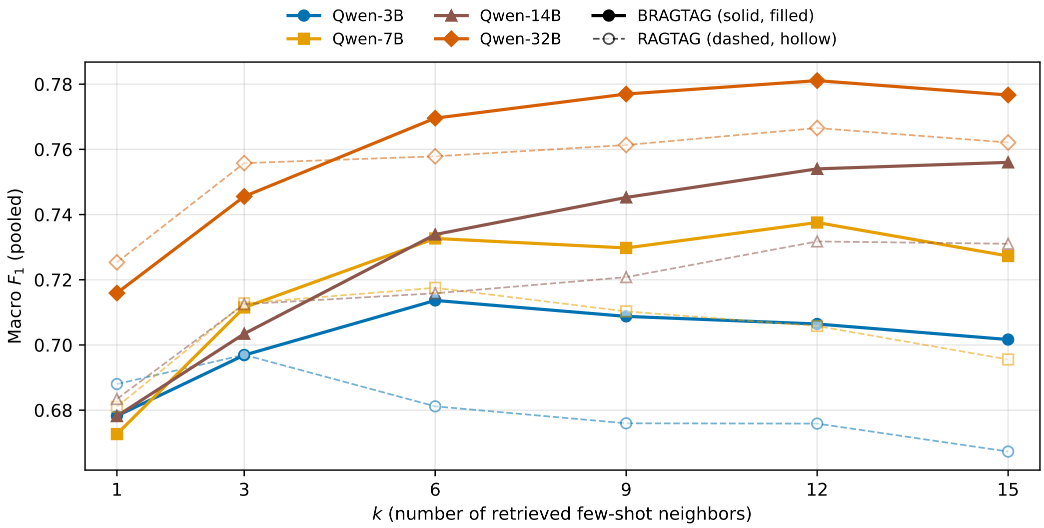


**Figure 5** Macro $F_1$ as a function of $k$ across all model sizes for both RAGTAG and BRAGTAG (project-specific setting)

propose a more balanced retrieval technique as an intervention to reduce question→bug misclassifications. To prevent damaging performance on true-bug issues, we apply this neighbor balancing only when the query is suspected to be a question. We calibrate this signal without including the test set. From each project's total 300 training issues, we assign 30 issues stratified by label as queries and 270 as the retrieval index, mirroring the PS setting. On this validation split, true-question queries retrieve almost balanced bug and question examples: averaged across $k \in [1, 15]$, the mean of $N_{\text{bug}} - N_{\text{question}}$ is $-1.36$. We therefore treat a query as a suspected question when the question count in the top-$k$ is within a small margin $m$ of the bug count ($N_{\text{bug}} - N_{\text{question}} \leq m$). Sweeping $m \in \{1, 2, 3, 4, 5\}$[4] on the validation split and averaging across $k \in [1, 15]$, $m$=3 had a more favorable trade-off against the other values, firing for 89.1% of true-question and 56.7% of true-bug queries. We adopt $m$=3 and refer to the resulting method as **B**alanced RAGTAG (BRAGTAG).

## 5.3 RQ3. Can a balanced retrieval intervention for retrieval-augmented few-shot prompting boost IRC performance?

Since the PS setting produced the best results for RAGTAG, we use PS as the default for evaluating BRAGTAG on the same $k \in \{1, 3, 6, 9, 12, 15\}$ grid. Figure 5 shows macro F1 vs $k$ for both methods across all four model sizes. BRAGTAG outperforms RAGTAG at every $k \geq 6$. At smaller $k \in \{1, 3\}$, BRAGTAG slightly underperforms by 0.001–0.010 macro F1, because the trigger fires too often and removes every retrieved example, practically reverting the query to zero-shot (40% of queries at $k$=1 and 18% at $k$=3).

BRAGTAG's best $k$ almost always increases compared to RAGTAG: Qwen-3B from $k$=3 to $k$=6, Qwen-7B from $k$=6 to $k$=12, Qwen-14B from $k$=12 to $k$=15, and Qwen-32B unchanged at $k$=12. When the trigger fires, BRAGTAG removes all bug examples from the

[4] The margin cannot be large given the small typical gap between bug and question neighbor counts for a true-question.

**Table 1** RAGTAG vs BRAGTAG at each model's best $k$ (PS, pooled).

| Model | Method | $k^*$ | Macro $F_1$ | $F_1^{\text{bug}}$ | $F_1^{\text{feat}}$ | $F_1^{\text{q}}$ | Invalid rate |
|---|---|---|---|---|---|---|---|
| Qwen-3B | RAGTAG | 3 | 0.697 | 0.711 | 0.803 | 0.577 | 0.2% |
| | BRAGTAG | 6 | 0.714 | 0.693 | 0.803 | 0.645 | 1.8% |
| Qwen-7B | RAGTAG | 6 | 0.718 | 0.731 | 0.816 | 0.605 | 3.5% |
| | BRAGTAG | 12 | 0.738 | 0.742 | 0.805 | 0.666 | 3.8% |
| Qwen-14B | RAGTAG | 12 | 0.732 | 0.735 | 0.838 | 0.622 | 4.5% |
| | BRAGTAG | 15 | 0.756 | 0.745 | 0.840 | 0.683 | 4.7% |
| Qwen-32B | RAGTAG | 12 | 0.767 | 0.759 | 0.842 | 0.699 | 4.6% |
| | BRAGTAG | 12 | 0.781 | 0.775 | 0.840 | 0.729 | 4.0% |

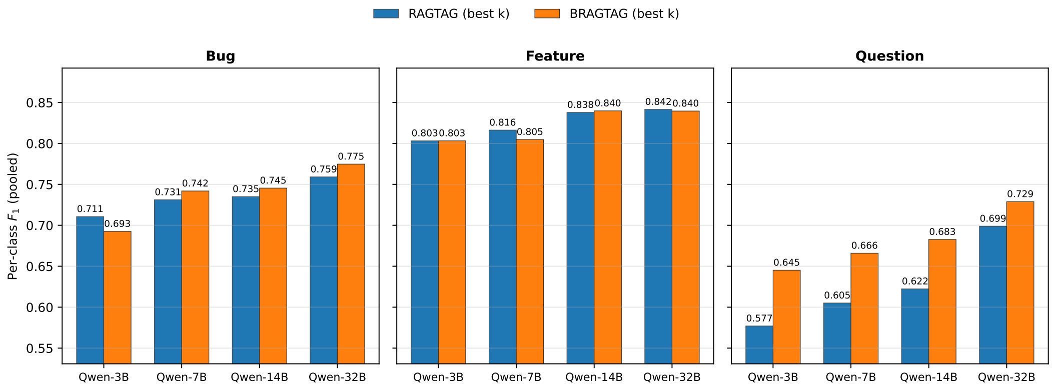


**Figure 6** Per-class $F_1$ at each method's best $k$ across all model sizes for RAGTAG and BRAGTAG.

top-$k$. Therefore, A higher initial $k$ is needed to leave enough remaining context for the LLM to use. At each method's best $k$, BRAGTAG outperforms RAGTAG on every model. The paired bootstrap 95% CI on the (BRAGTAG − RAGTAG) macro $F_1$ difference excludes zero in all four cells:[5] Qwen-3B +0.017 [+0.003, +0.031], Qwen-7B +0.020 [+0.007, +0.032], Qwen-14B +0.024 [+0.014, +0.034], and Qwen-32B +0.014 [+0.007, +0.022]. Table 1 reports the full best configuration comparison.

The performance gain comes from an increase on the question and bug classes with minimal impact on feature. Figure 6 shows per-class $F_1$ at each method's best $k$. Question $F_1$ improves by 0.030–0.068 across all four model sizes. Bug $F_1$ stays within 0.018 of RAGTAG, with a slight loss for Qwen-3B and slight gains for the larger models. Therefore, removing bug neighbors in some cases did not decrease overall bug performance. Feature $F_1$ is essentially unchanged. The question→bug misclassification rate drops: at each model's best $k$, RAGTAG misclassifies 26–36% of true-question issues as bugs while BRAGTAG reduces this to 19–27%. Averaged across $k \in [1, 15]$, the rate drops from 31–40% (RAGTAG) to 24–36% (BRAGTAG). Question→feature misclassification is unaffected. This confirms that BRAGTAG's improvement comes from the targeted reduction of question→bug misclassification.

Outputs from both approaches that failed to parse to a valid label are marked as *invalid* and counted as incorrect. Averaged across $k \in [1, 15]$, BRAGTAG has a lower invalid rate than RAGTAG (2.3% vs 3.0% across the four models).

[5] 1,000 paired bootstrap resamples per model.

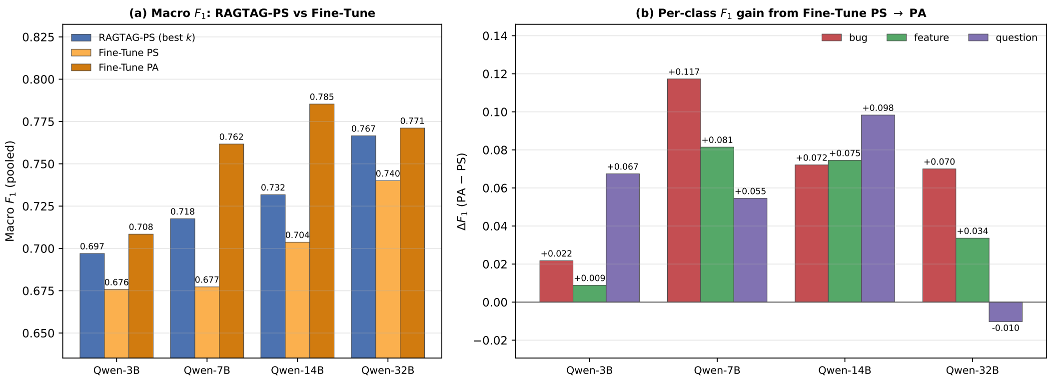


**Figure 7** **(a)** Macro $F_1$ for RAGTAG at its best $k$ in PS vs Fine-Tune in PS and PA across all four model sizes. **(b)** Per-class $F_1$ gain from Fine-Tune PS to Fine-Tune PA ($\Delta F_1$ = PA − PS) for each class and model size.

**Table 2** RAGTAG, BRAGTAG, and Fine-Tune at each method's best configuration (RAGTAG/BRAGTAG at their best $k$ on PS, Fine-Tune on PA). Macro $F_1$ and per-class $F_1$ are computed on raw predictions. The +VOTAG column reports macro $F_1$ with VOTAG as fallback for the invalid LLM outputs. Train and Inference are run-times on a single GPU with Total as their sum. RAM is the peak GPU memory observed

| Model | Method | $k^*$ | Macro $F_1$ | +VOTAG | $F_1^{\text{bug}}$ | $F_1^{\text{feat}}$ | $F_1^{\text{q}}$ | RAM (GB) | Train (h) | Infer (h) | Total (h) |
|---|---|---|---|---|---|---|---|---|---|---|---|
| Qwen-3B | RAGTAG | 3 | 0.697 | 0.696 | 0.711 | 0.803 | 0.577 | 2.9 | – | 0.18 | 0.18 |
| | BRAGTAG | 6 | 0.714 | 0.720 | 0.693 | 0.803 | 0.645 | 2.9 | – | 0.22 | 0.22 |
| | Fine-Tune | – | 0.708 | 0.711 | 0.736 | 0.790 | 0.599 | 5.5 | 0.31 | 0.11 | 0.42 |
| Qwen-7B | RAGTAG | 6 | 0.718 | 0.729 | 0.731 | 0.816 | 0.605 | 6.8 | – | 0.50 | 0.50 |
| | BRAGTAG | 12 | 0.738 | 0.751 | 0.742 | 0.805 | 0.666 | 6.8 | – | 0.60 | 0.60 |
| | Fine-Tune | – | 0.762 | 0.762 | 0.762 | 0.846 | 0.678 | 9.9 | 0.48 | 0.10 | 0.58 |
| Qwen-14B | RAGTAG | 12 | 0.732 | 0.746 | 0.735 | 0.838 | 0.622 | 12.6 | – | 1.37 | 1.37 |
| | BRAGTAG | 15 | 0.756 | 0.771 | 0.745 | 0.840 | 0.683 | 12.6 | – | 1.35 | 1.35 |
| | Fine-Tune | – | 0.785 | 0.785 | 0.792 | 0.828 | 0.736 | 16.7 | 1.49 | 0.22 | 1.71 |
| Qwen-32B | RAGTAG | 12 | 0.767 | 0.779 | 0.759 | 0.842 | 0.699 | 22.3 | – | 4.62 | 4.62 |
| | BRAGTAG | 12 | 0.781 | 0.792 | 0.775 | 0.840 | 0.729 | 22.3 | – | 4.15 | 4.15 |
| | Fine-Tune | – | 0.771 | 0.771 | 0.791 | 0.853 | 0.669 | 31.5 | 2.28 | 0.44 | 2.71 |

## 5.4 RQ4. Can retrieval-augmented few-shot prompting match LoRA fine-tuning for IRC, and what are the costs of each approach?

### 5.4.0.1 Classification Performance.

We fine-tune all models in both PA and PS settings. Figure 7(a) compares macro $F_1$ for fine-tuning in both settings against RAGTAG's PS at each model's best $k$. Fine-tune PA outperforms PS at every model size, by +0.033 (Qwen-3B), +0.084 (Qwen-7B), +0.082 (Qwen-14B), and +0.031 (Qwen-32B) macro $F_1$. This trend matches prior findings on LLM fine-tuning for IRC [22].

RAGTAG's PS outperforms Fine-Tune's PS at every model size, by +0.021 (Qwen-3B) to +0.040 (Qwen-7B) macro $F_1$. This shows that using only 300 labeled issues per project, retrieval-augmented few-shot is more effective than LoRA fine-tuning for IRC.

Figure 7(b) shows where the additional PA training data helps most in fine-tuning. Averaged across all four models, bug $F_1$ gains +0.070 going from PS to PA, question gains +0.053, and feature gains +0.050. Qwen-32B is the only exception: its question $F_1$ drops by 0.010 while bug gains +0.070.

Table 2 reports macro and per-class $F_1$ at each method's best configuration. RAGTAG has competitive macro $F_1$ with fine-tuning: aggregated across the four model sizes (13,200 paired predictions), fine-tune leads RAGTAG by 0.029 macro $F_1$ (paired bootstrap 95% CI $[-0.037, -0.022]$). RAGTAG is closer to fine-tune at the smallest and largest models and farther in the middle, with per-model (RAGTAG − fine-tune) macro $F_1$ deltas (paired bootstrap 95% CIs in brackets) of $-0.011$ $[-0.028, +0.007]$ (Qwen-3B), $-0.044$ $[-0.059, -0.029]$ (Qwen-7B), $-0.053$ $[-0.068, -0.039]$ (Qwen-14B), and $-0.004$ $[-0.020, +0.009]$ (Qwen-32B). RAGTAG and fine-tune are statistically equal at Qwen-3B and Qwen-32B.

BRAGTAG's balanced retrieval narrows this gap: the aggregate (BRAGTAG − fine-tune) difference is $-0.010$ with paired bootstrap 95% CI $[-0.018, -0.003]$, passing TOST equivalence at $\delta$=0.02.[6] Similar to RAGTAG, BRAGTAG performs better relative to fine-tune at the smallest and largest models: BRAGTAG is statistically tied with fine-tune at Qwen-3B and Qwen-32B while fine-tune leads on the mid-sized ones, with per-model (BRAGTAG − fine-tune) deltas of $+0.006$ $[-0.012, +0.024]$ (Qwen-3B), $-0.024$ $[-0.039, -0.009]$ (Qwen-7B), $-0.029$ $[-0.044, -0.013]$ (Qwen-14B), and $+0.010$ $[-0.005, +0.024]$ (Qwen-32B).

BRAGTAG has the best question $F_1$ on Qwen-3B and Qwen-32B, while fine-tune leads on Qwen-7B and Qwen-14B. Fine-tune leads on bug across all four model sizes; on feature, BRAGTAG wins Qwen-3B and Qwen-14B while fine-tune wins Qwen-7B and Qwen-32B. These results show that BRAGTAG is able to compete with fine-tune due to its targeted question→bug correction, boosting question $F_1$ without sacrificing bug or feature. Averaged across the four models, BRAGTAG is also the most class-balanced approach, with the lowest per-class $F_1$ standard deviation (0.058 vs 0.067 for fine-tune and 0.082 for RAGTAG) and the highest worst-class $F_1$ floor (0.681 vs 0.671 and 0.626).

Fine-tune produces fewer invalid outputs than RAGTAG and BRAGTAG in both settings, because the model learns the output format during training. Averaged across the four models, fine-tune's invalid rate is 0.28% in PA and 0.38% in PS, below RAGTAG's 3.0% and BRAGTAG's 2.3% (averaged across $k \in [1, 15]$). In an ideal triage scenario, every issue report receives a label, so a known limitation of LLMs is the production of outputs that do not comply with the instructions [40, 2]. To compensate for the invalid predictions we propose using VOTAG as a fallback mechanism. VOTAG never produces an invalid label and uses the neighbors already retrieved for RAGTAG/BRAGTAG with no LLM inference cost. We apply the fallback to all three methods at their best data scopes for fair comparison: VOTAG-PS for RAGTAG/BRAGTAG and VOTAG-PA for fine-tune. Under this protocol, BRAGTAG's reaches statistical equivalence with fine-tune, passing TOST at $\delta$=0.01 (bootstrap 95% CI $[-0.007, +0.008]$).

#### 5.4.0.2 Computation and Data Cost.

The methods have different hardware requirements. Figure 8(a) shows peak GPU RAM by model. RAGTAG and BRAGTAG require identical GPU memory that grows with model size, while fine-tune requires more at every size due to training overheads such as, gradients, optimizer state, and activations retained for the backward pass. Averaged across the model sizes, RAGTAG and BRAGTAG take 33% less GPU memory than fine-tune (47%, 31%, 25%, and 29% less at Qwen-3B, 7B, 14B, and 32B respectively).

Figure 8(b) shows total GPU time at each model size. Fine-tune takes longer than RAGTAG at Qwen-3B, 7B, and 14B. But at Qwen-32B fine-tune (2.71 h) takes less time than

[6] 1,000 paired bootstrap resamples per model for both RAGTAG vs fine-tune and BRAGTAG vs fine-tune comparisons.

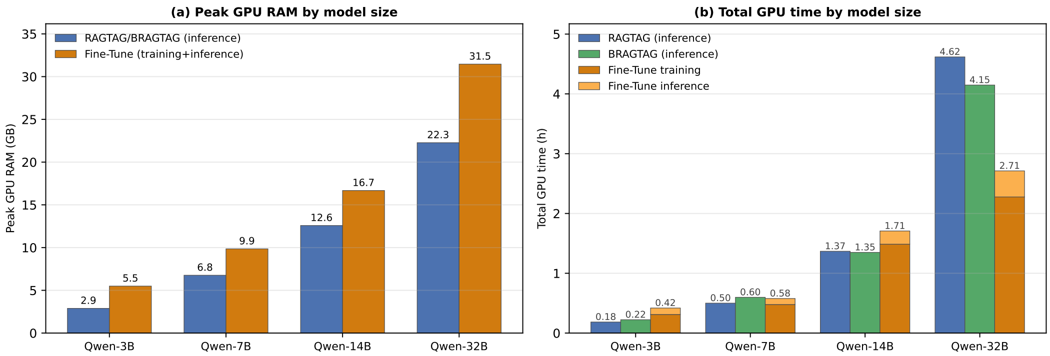


**Figure 8** **(a)** Peak GPU RAM by model size: RAGTAG/BRAGTAG inference vs Fine-Tune training + inference. **(b)** Total GPU time by model size: RAGTAG and BRAGTAG vs Fine-Tune

both RAGTAG (4.62 h) and BRAGTAG (4.15 h). Inference time on each model naturally grows with prompt length. Fine-tune processes only the query issue, while RAGTAG and BRAGTAG process significantly more context due to the additional few-shot examples per query. As a result, fine-tune inference takes much less time than rag-based inference, and fine-tune's total run-time is dominated by training. At Qwen-7B and Qwen-32B, BRAGTAG's inference cost alone exceeds fine-tune's combined training and inference time. BRAGTAG's intervention shortens the average context enough at Qwen-14B and Qwen-32B for BRAGTAG to run less than RAGTAG. However, BRAGTAG runs longer at Qwen-3B and Qwen-7B because the intervention could not compensate for the higher best $k$ value.

Notably, RAGTAG and BRAGTAG in PS retrieve only from 300 labeled issues per project, while fine-tune PA trains on the full 3,300 issues across all 11 projects to reach its best configuration. Therefore, at 11× less labeled data per project, RAGTAG remains competitive with fine-tune, and BRAGTAG closes the gap to TOST equivalence at $\delta$=0.02, tightening to statistical equivalence at $\delta$=0.01 under the VOTAG fallback for invalid LLM outputs.

# 6 Discussion

## 6.1 Failure Analysis

In this section we conduct a manual qualitative analysis to better understand the misclassification patterns across the explored IRC methods. We inspect the misclassifications produced by Qwen-32B (the strongest model) under each method's best configuration. From all the misclassifications across RAGTAG, BRAGTAG, and fine-tune, we drew a random sample of 50 misclassifications per method (150 total). Each Method's sample includes 30 true-question, 10 true-bug, and 10 true-feature errors. Each case was analyzed by two authors independently, with discrepancies resolved through discussions. The observed failure categories and their frequencies are as follows:

***Wrong template format (77.33%).*** Issue reports whose content and ground-truth label contradicts their chosen template and structure. All of these errors are from questions and feature requests that the user filed under a bug-report template (*Steps to reproduce*, *Expected*, *Actual*, *System Information*). Consequently, the structure of the bug template misleads the LLM classifier towards predicting bug, regardless of the issue's content. For example, `microsoft/vscode#193985`[7] is an actual support question that opens with a literal `Type:Bug` because it uses VS Code's bug template.

***Hybrid intent (13.33%).*** Issues that could have multiple defensible labels due to their combined intents. These errors mostly take place when a fix or a feature request is layered in a question or bug. For example, in `kubernetes/kubernetes#120364`[8] the author both reports a concrete bug and in the same body explicitly requests a new feature: "*Please, give us multiarchitecture image! and fix monoarchitecture images too!*"

***Label noise (9.33%).*** Issues whose ground-truth label does not match its content. For example, `tensorflow/tensorflow#61710`[9] is titled "*Will there a MLP model in the future version?*" which requests a new API feature for building MLPs. Yet the dataset labels it as a bug.

We also sampled 30 issues with *invalid* predictions per method (60 total) and observed three failure modes. In roughly 73% of cases, the model either continued to generate issue-body content or its own reasoning. Another 23% were hallucinations such as repeated digits and punctuations. The remaining 3% are parsable labels that fall outside the three valid classes (e.g., `documentation`).

## 6.2 Key Insights and Practical Implications

**The practicality of context retrieval for IRC.** Our results indicate that RAG-based approaches offer a more practical solution for IRC than fine-tuning. With 11× less labeled data per project and 25–47% less peak GPU memory, RAGTAG performs competitively against fine-tuning (trails by 0.029 in aggregate macro F1), and BRAGTAG closes the gap to TOST equivalence at $\delta$=0.02, tightening to statistical equivalence at $\delta$=0.01 under the VOTAG fallback mechanism for invalid LLM outputs (Section 5.4). RAG-based methods offer deployment advantages over fine-tuning as well: adding a new labeled issue report becomes an incremental index update rather than a retraining cycle, and the model can be swapped in production with no project-specific retraining. For extremely resource-constrained scenarios, even VOTAG PS is a defensible baseline: it reaches macro $F_1 = 0.595$ ($k$=15), within 0.018 of Qwen-3B's zero-shot 0.613 (Section 5.1, Section 5.2), at a fraction of the GPU cost (≈240 MB vs 2.9 GB).

**Combining classification and validation.** Our failure analysis (Section 6.1) shows that users frequently file support questions and feature requests under the bug-report template, which can bias the classifier towards bug regardless of the content. This emphasizes the importance of refining the new LLM-assisted triage pipelines now in place on platforms such as GitHub [19] and GitLab [20]. In these pipelines, the reporter describes the report to an LLM, which then suggest the correct template and labels. Report validation is also important since mislabeling and duplication is a common problem among contributors[3, 37]. Studies have already explored validation for bug reports [30, 46, 1, 15]. However, combining classification and validation in a cohesive workflow is underexplored.

## 6.3 Future Work

**Generalization to more diverse environments.** Our analysis covers a large benchmark across 11 GitHub projects with a balanced 600 issues per project. We encourage future work

[7] https://github.com/microsoft/vscode/issues/193985
[8] https://github.com/kubernetes/kubernetes/issues/120364
[9] https://github.com/tensorflow/tensorflow/issues/61710

to evaluate IRC methods under more diverse settings such as imbalanced label distributions with one or more minority classes, multi-label settings where more than one label can be predicted for a single issue, and other issue tracking platforms (e.g., Jira, Bugzilla). These future directions will assess the generalizability of our findings and uncover new patterns.

**Other retrieval techniques.** Our study empirically shows that retrieval heuristics can soften model biases and even match fine-tuning without parameter updates. This opens the door for future research on other retrieval-side interventions for different IRC settings. A concrete future direction is a two-stage inference scheme where a first LLM call filters or re-ranks the retrieved neighbors based on their relevance, and a second call performs the classification. Settings such as embedding model and similarity metric are also worth exploring in future work, because of their impact on retrieval quality.

**Prompt tuning** Our proposed few-shot prompt structure is one of many reasonable options. Since other formats can yield better results, we encourage future work to explore other structures and advanced strategies such as Chain-of-Thought (CoT) prompting [31].

**Other LLM families.** Our study uses the Qwen2.5-Instruct family at four sizes to isolate model scale from architecture (Section 4.2). Now that the competitiveness of retrieval-augmented few-shot prompting is empirically established on this family, we encourage future work to conduct a dedicated study identifying which open-source LLM families are most effective for retrieval-augmented few-shot IRC and characterizing how architectural choices affect retrieval-augmented classification.

## 7 Threats to Validity

#### 7.0.0.1 Internal validity

**BRAGTAG's margin.** Empirical selection of BRAGTAG's margin can potentially be an internal threat, since the value could be specifically tuned for the models and test data in this study. To mitigate this, margin calibration used only the training split and did not involve the test split (Section 5.2). Furthermore, the same fixed margin is used across all models and projects with no per-project or per-model tuning. Finally, the selected margin in this study is not framed as a universally optimal constant. Any project can re-derive the margin with negligible cost by only using the retrieval index of available historical issue reports.

**Hyperparameters and prompt format.** Different hyperparameters and prompts can change model performance. We address this when reproducing the fine-tuning baseline in this study by adopting the same hyperparameters and prompt formats used in prior work [22, 5].

#### 7.0.0.2 External Validity

**Project generalizability.** Issue report text varies in writing style, vocabulary, and structure between software projects [2], which can be a potential threat to the generalizability of our findings. To mitigate this, we used a large benchmark of 6,600 issues spanning 11 different projects, each with their own reporting conventions and writing styles.

**Generalizability across settings.** The primary aim of this work is to empirically assess the effectiveness of retrieval-augmented few-shot prompting versus LoRA fine-tuning for IRC and to explore how results vary with different model scales. This goal required exploring a large variable space across model sizes, $k$ values, data scopes, and different retrieval-based methods. However, Many configurations — including LLM architecture, prompt structure, embedding model, and retrieval technique — can each lead to different results, which can be a potential threat to the generalizability of our findings beyond the settings used in this study. Recognizing this threat, we encourage future work to build on our findings and conduct dedicated studies under different settings. To facilitate such studies, we designed our experiment pipeline in a modular way that allows all configurations to be easily substituted.

#### 7.0.0.3 Construct Validity

**LLM randomness.** The reported performance metrics can vary between different runs due to LLM randomness. To mitigate this variance, we ran the experiments three times and report the average.

## 8 Conclusion

**Method.** We evaluate all three against a LoRA fine-tuning baseline on an 11-project benchmark, in both *project-specific* (PS) and *project-agnostic* (PA) data scope settings. Experiments use Qwen2.5-Instruct at four sizes.

In this study, we investigated retrieval-augmented few-shot prompting as a cost-efficient alternative to LoRA fine-tuning for issue report classification (IRC). Given a query issue, we retrieved its top-$k$ nearest neighbors from historical issue reports to be used in three proposed methods. First, RAGTAG, which presents the neighbors as labeled examples in a few-shot prompt. Second, BRAGTAG, a RAGTAG variant that drops bug-labeled neighbors when the retrieved label distribution indicates the query is likely a question. Third, VOTAG, which predicts the label by a weighted vote on the neighbors' labels, establishing a non-parametric baseline.

We evaluated all three methods against the LoRA fine-tuning baseline using the Qwen2.5-Instruct model at four sizes. Evaluations were conducted under both project-specific (PS) and project-agnostic (PA) data scope settings.

The results show that retrieval-augmented few-shot prompting is a practical alternative to fine-tuning for IRC. RAGTAG PS outperforms fine-tuning PS by 0.021–0.040 macro $F_1$ across all four model sizes, and trails fine-tuning PA by 0.029 macro $F_1$ in aggregate despite using 11× less labeled data per project. BRAGTAG PS closes this gap to TOST equivalence at $\delta$=0.02, tightening to statistical equivalence at $\delta$=0.01 under the VOTAG fallback protocol for invalid LLM outputs. Averaged across all models, retrieval-augmented few-shot also required 33% less peak GPU memory than fine-tuning.

These findings position retrieval-augmented few-shot prompting as the more practical IRC method, particularly when hardware is constrained, labeled data is limited, or deployment requires frequent model or data updates. We encourage future work to evaluate these methods in diverse data distribution scenarios and to explore additional retrieval techniques and configurations.

## 9 Data Availability

### References


1 Toufique Ahmed, Jatin Ganhotra, Rangeet Pan, Avraham Shinnar, Saurabh Sinha, and Martin Hirzel. Otter: Generating tests from issues to validate swe patches. *arXiv preprint arXiv:2502.05368*, 2025.

2 Arshia Akhavan, Alireza Hoseinpour, Abbas Heydarnoori, Hamid Bagheri, and Mehdi Keshani. Linkanchor: An autonomous llm-based agent for issue-to-commit link recovery, 2026. URL: `https://arxiv.org/abs/2508.12232`, `arXiv:2508.12232`.

3 Giuliano Antoniol, Kamel Ayari, Massimiliano Di Penta, Foutse Khomh, and Yann-Gaël Guéhéneuc. Is it a bug or an enhancement? a text-based approach to classify change requests. In *Proceedings of the 2008 conference of the center for advanced studies on collaborative research: meeting of minds*, pages 304–318, 2008.

4 Gabriel Aracena, Kyle Luster, Fabio Santos, Igor Steinmacher, and Marco A. Gerosa. Applying large language models api to issue classification problem, 2024. URL: `https://arxiv.org/abs/2401.04637`, `arXiv:2401.04637`.

5 Gabriel Aracena, Kyle Luster, Fabio Santos, Igor Steinmacher, and Marco A Gerosa. Applying large language models to issue classification: Revisiting with extended data and new models. *Science of Computer Programming*, page 103333, 2025.

6 Maram Assi, Safwat Hassan, and Ying Zou. Llm-cure: Llm-based competitor user review analysis for feature enhancement. *ACM Transactions on Software Engineering and Methodology*, 35(4):1–25, 2026.

7 Shikhar Bharadwaj and Tushar Kadam. Github issue classification using bert-style models. In *Proceedings of the 1st International Workshop on Natural Language-based Software Engineering*, pages 40–43, 2022.

8 Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

9 Jordi Cabot, Javier Luis Cánovas Izquierdo, Valerio Cosentino, and Belén Rolandi. Exploring the use of labels to categorize issues in open-source software projects. In *2015 IEEE 22nd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*, pages 550–554. IEEE, 2015.

10 Gemma Catolino, Fabio Palomba, Andy Zaidman, and Filomena Ferrucci. Not all bugs are the same: Understanding, characterizing, and classifying bug types. *Journal of Systems and Software*, 152:165–181, 2019.

11 Giuseppe Colavito, Filippo Lanubile, and Nicole Novielli. Issue report classification using pre-trained language models. In *Proceedings of the 1st International Workshop on Natural Language-based Software Engineering*, pages 29–32, 2022.

12 Giuseppe Colavito, Filippo Lanubile, and Nicole Novielli. Benchmarking large language models for automated labeling: The case of issue report classification. *Information and Software Technology*, page 107758, 2025.

13 Giuseppe Colavito, Filippo Lanubile, Nicole Novielli, and Luigi Quaranta. Leveraging gpt-like llms to automate issue labeling. In *Proceedings of the 21st International Conference on Mining Software Repositories*, pages 469–480, 2024.

14 Thomas Cover and Peter Hart. Nearest neighbor pattern classification. *IEEE transactions on information theory*, 13(1):21–27, 1967.

15 Emre Dinç and Eray Tüzün. Judge the votes: A system to classify bug reports and give suggestions. In *2025 2nd IEEE/ACM International Conference on AI-powered Software (AIware)*, pages 01–10. IEEE, 2025.

16 Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The faiss library. *IEEE Transactions on Big Data*, 2025.

17 Sahibsingh A Dudani. The distance-weighted k-nearest-neighbor rule. *IEEE Transactions on Systems, Man, and Cybernetics*, (4):325–327, 1976.

18 Qiang Fan, Yue Yu, Gang Yin, Tao Wang, and Huaimin Wang. Where is the road for issue reports classification based on text mining? In *2017 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, pages 121–130. IEEE, 2017.

19 GitHub. GitHub Copilot. `https://github.com/features/copilot`, 2025.

20 GitLab. GitLab Duo. `https://about.gitlab.com/gitlab-duo/`, 2025.

**21** Luiz Gomes, Ricardo da Silva Torres, and Mario Lúcio Côrtes. Bert-and tf-idf-based feature extraction for long-lived bug prediction in floss: A comparative study. *Information and Software Technology*, 160:107217, 2023.

**22** Jueun Heo and Seonah Lee. A study on applying large language models to issue classification. In *2025 IEEE/ACM 33rd International Conference on Program Comprehension (ICPC)*, pages 1–11. IEEE Computer Society, 2025.

**23** Kim Herzig, Sascha Just, and Andreas Zeller. It's not a bug, it's a feature: how misclassification impacts bug prediction. In *2013 35th international conference on software engineering (ICSE)*, pages 392–401. IEEE, 2013.

**24** Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Liang Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. *Iclr*, 1(2):3, 2022.

**25** Huihui Huang, Ratnadira Widyasari, Ting Zhang, Ivana Clairine Irsan, Jieke Shi, Han Wei Ang, Frank Liauw, Eng Lieh Ouh, Lwin Khin Shar, Hong Jin Kang, et al. Back to the basics: Rethinking issue-commit linking with llm-assisted retrieval. *arXiv preprint arXiv:2507.09199*, 2025.

**26** Maliheh Izadi. Catiss: An intelligent tool for categorizing issues reports using transformers. In *Proceedings of the 1st international workshop on natural language-based software engineering*, pages 44–47, 2022.

**27** Maliheh Izadi, Kiana Akbari, and Abbas Heydarnoori. Predicting the objective and priority of issue reports in software repositories. *Empirical Software Engineering*, 27(2):50, 2022.

**28** Gaeul Jeong, Sunghun Kim, and Thomas Zimmermann. Improving bug triage with bug tossing graphs. In *Proceedings of the 7th joint meeting of the European software engineering conference and the ACM SIGSOFT symposium on The foundations of software engineering*, pages 111–120, 2009.

**29** Rafael Kallis, Andrea Di Sorbo, Gerardo Canfora, and Sebastiano Panichella. Ticket tagger: Machine learning driven issue classification. In *2019 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 406–409. IEEE, 2019.

**30** Lara Khatib, Noble Saji Mathews, and Meiyappan Nagappan. Assertflip: Reproducing bugs via inversion of llm-generated passing tests. *arXiv preprint arXiv:2507.17542*, 2025.

**31** Anil Koyuncu. Exploring fine-grained bug report categorization with large language models and prompt engineering: An empirical study. *ACM Transactions on Software Engineering and Methodology*, 2025.

**32** Van-Hoang Le and Hongyu Zhang. Log parsing with prompt-based few-shot learning. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pages 2438–2449. IEEE, 2023.

**33** Zhifang Liao, Dayu He, Zhijie Chen, Xiaoping Fan, Yan Zhang, and Shengzong Liu. Exploring the characteristics of issue-related behaviors in github using visualization techniques. *IEEE Access*, 6:24003–24015, 2018.

**34** Jiachang Liu, Dinghan Shen, Yizhe Zhang, William B Dolan, Lawrence Carin, and Weizhu Chen. What makes good in-context examples for gpt-3? In *Proceedings of Deep Learning Inside Out (DeeLIO 2022): The 3rd workshop on knowledge extraction and integration for deep learning architectures*, pages 100–114, 2022.

**35** Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the association for computational linguistics*, 12:157–173, 2024.

**36** Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.

**37** Nitish Pandey, Debarshi Kumar Sanyal, Abir Hudait, and Amitava Sen. Automated classification of software issue reports using machine learning techniques: an empirical study. *Innovations in Systems and Software Engineering*, 13(4):279–297, 2017.

38 Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. *arXiv preprint arXiv:1908.10084*, 2019.

39 Fabio Santos, Joseph Vargovich, Bianca Trinkenreich, Italo Santos, Jacob Penney, Ricardo Britto, João Felipe Pimentel, Igor Wiese, Igor Steinmacher, Anita Sarma, et al. Tag that issue: applying api-domain labels in issue tracking systems. *Empirical Software Engineering*, 28(5):116, 2023.

40 Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. Quantifying language models' sensitivity to spurious features in prompt design or: How i learned to start worrying about prompt formatting. *arXiv preprint arXiv:2310.11324*, 2023.

41 Igor Steinmacher, Christoph Treude, and Marco Aurelio Gerosa. Let me in: Guidelines for the successful onboarding of newcomers to open source projects. *IEEE Software*, 36(4):41–49, 2018.

42 Alexander Trautsch and Steffen Herbold. Predicting issue types with sebert. In *Proceedings of the 1st international workshop on natural language-based software engineering*, pages 37–39, 2022.

43 Joseph Vargovich, Fabio Santos, Jacob Penney, Marco A Gerosa, and Igor Steinmacher. Givemelabeledissues: An open source issue recommendation system. In *2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*, pages 402–406. IEEE, 2023.

44 Julian Von der Mosel, Alexander Trautsch, and Steffen Herbold. On the validity of pre-trained transformers for natural language processing in the software engineering domain. *IEEE Transactions on Software Engineering*, 49(4):1487–1507, 2022.

45 Jun Wang, Xiaofang Zhang, and Lin Chen. How well do pre-trained contextual language representations recommend labels for github issues? *Knowledge-Based Systems*, 232:107476, 2021.

46 Xinchen Wang, Pengfei Gao, Xiangxin Meng, Chao Peng, Ruida Hu, Yun Lin, and Cuiyun Gao. Aegis: An agent-based framework for general bug reproduction from issue descriptions. *arXiv preprint arXiv:2411.18015*, 2024.

47 Jason Wei, Maarten Bosma, Vincent Y Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M Dai, and Quoc V Le. Finetuned language models are zero-shot learners. *arXiv preprint arXiv:2109.01652*, 2021.

48 Hongrun Wu, Yutao Ma, Zhenglong Xiang, Chen Yang, and Keqing He. A spatial–temporal graph neural network framework for automated software bug triaging. *Knowledge-Based Systems*, 241:108308, 2022.

49 An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.